Guide Ivy
Specification-based testing of QUIC

Crochet Christophe
Computer Science dep.
Université catholique de Louvain
Louvain-la-Neuve, Belgium
christophe.crochet@student.uclouvain.be

Sambon Jean-François
Computer Science dep.
Université catholique de Louvain
Louvain-la-Neuve, Belgium
jean-francois.sambon@student.uclouvain.be

***Abstract*—This document is a model and instructions for the specification-based language Ivy [1] create by *Kenneth McMillan* [6], *Odel Padon*, *Aurojit Panda* [2], *Mooly Sagiv* [3] and *Sharon Shoham* [7] applied to QUIC topic, that we used in our master thesis "*Toward-verification-of-QUIC-extensions*" subject proposed by *Bonaventure Olivier* [5] and *Legay Axel* [4] (and help of the networking team of UCL of course)**

***Index Terms*—QUIC, Ivy, Specification, Test, Network, Protocol, Verification**

I. Introduction

In this document, we will explain you the main Ivy concepts which are used for the specification-based testing of QUIC implementations.

It will be based on the official documentation of Ivy where we will include some specific examples to point out the concept presented for the 1.7 version of Ivy. We will only present a subset of Ivy functionalities and encourage the reader to read the official documentation for more in depth explanation.

In our thesis, we are checking if Ivy is in practice usable for protocols testing first and more specifically if it can be used to test protocol extensions with a focus on QUIC. Therefore, The goal of this document is to explain more in depth how Ivy is working from behind and how can we use this tool.

II. Ivy

"IVy is a research tool intended to allow interactive development of protocols and their proofs of correctness and to provide a platform for developing and experimenting with automated proof techniques. In particular, IVy provides interactive visualization of automated proofs, and supports a use model in which the human protocol designer and the automated tool interact to expose errors and prove correctness." [15] Can extract executable programs by translation with python to C++ for efficiency. [16]

First of all, Ivy language has three kinds of object:

1) **Data values** which are the variables that we are going to store the state of our program.
2) **Function values** which are mathematical function : from a given input it gives always the same output and does not modify the environment.
3) **Procedure** can also return a result (or not) but will modify the environment which allow different results for different execution.

We can also use high-order programming with the Ivy language by putting into variables data values and function values. There are no references in Ivy, every variable is link to a different object.

The language is synchronous, which means that all actions occur in reaction to change in the environment and that they are also all isolated, so they appear to occur instantly.

These features are present since most of verification problems such as invariant checking, model checking rely on a subset of first-order logic formulas: decidable fragments. [8] More details will be given in the decidability section of the report.

We will explain the Ivy concepts, and then exploring the specification of the different QUIC layer, to finally introduce some concepts of the Ivy solver:

1) Application over QUIC
2) TLS handshake security
3) QUIC Frame protocol
4) QUIC Packet protocol
5) QUIC Packet Protection for reliability
6) UDP

III. Ivy concepts used for QUIC

A. Type object, Action and State

```
object frame = {
    # The base type for frames
    type this
    # Stream frames
    object stream = {
        # Stream frames are a variant of frame
        variant this of frame = struct {
            off : bool, # is there an offset field
            len : bool, # is there a length field
            fin : bool, # is this the final offset
```

```
                id   : stream_id ,     # the stream ID
                offset : stream_pos ,# the stream offset
                length : stream_pos ,# length of the data
                data : stream_data   # the stream data
            }
        }
}
```

```
object frame = {
    ...
    action handle(f:this,    # param_name:type
                  scid:cid,
                  dcid:cid,
                  e:quic_packet_type) = {
       # this generic action should never be called
       require false;
    }
}
action enqueue_frame(scid:cid,
                     f:frame,
                     e:quic_packet_type,
                     probing:bool) = {
       # Code
}
```

```
object frame = {
    ...
    object stream = {
        ...
        action handle(f:this,
                      scid:cid,
                      dcid:cid,
                      e:quic_packet_type) = {
           require connected(dcid)
                 & connected_to(dcid)=scid;
           require e = quic_packet_type.one_rtt
                 & established_1rtt_keys(scid);
           call enqueue_frame(scid,f,e,false);
        }
    }
}
```

In this example, we can already see the most used concepts of Ivy for testing QUIC protocols. First, Ivy use the concept of `type objects` which have a **state** that can be manipulated through **action**.

This example also show you that type objects are define from a very abstract and generic point of view, here we start by defining a `frame` type object, and then we can define `variant` of the type object that set the state of more "embedded" structure, here with `stream frame`, and any other type of frame that QUIC define in the specification. The same idea is use through all layer of QUIC.

Action are also defined from an abstract and generic point of view and then adapted to specific case, here for the action `handle` which is associated to frame type object and then define this action for each type of frame but can also be global such as the action `enqueue_frame`. The code of the action `handle` mostly define a suite of requirements/conditions that should be true at each step to continue the execution, which could be defined in most part by safety properties of the specification ("must" statement) even if this is not always the case.

B. Modules

Modules are like template class in object oriented programming that can be instantiated. For QUIC, it is used for defining a general class of objects but can be used to capture more general theory and proof. Here is a example for the QUIC transport parameters:

```
type trans_params_struct
object transport_parameter = {
    type this
    action set(p:this,s:trans_params_struct)
        returns (s:trans_params_struct) = {}
}
module trans_params_ops(ptype) = {
    destructor is_set(S:trans_params_struct) : bool
    destructor value(S:trans_params_struct) : ptype
    action set(p:ptype,s:trans_params_struct)
        returns (s:trans_params_struct) = {
        is_set(s) := true;
        value(s) := p;
    }
}
```

```
object initial_max_stream_data_bidi_local = {
    variant this of transport_parameter = struct {
        stream_pos_32 : stream_pos  # tag = 0
    }
    instantiate trans_params_ops(this)
}
object initial_max_data = {
    variant this of transport_parameter = struct {
        stream_pos_32 : stream_pos # tag = 1
    }
    instantiate trans_params_ops(this)
}
```

In this example, we can see that the object can instantiate `trans_params_ops` automatically such that we can now check if a parameter is set or not and make call directly on transport parameter object with destructor which are used to say that if the transport parameter is not instantiated then the `is`$_{set}$ return `false` (we don't really have definition for that in the Ivy documentation, for now), action and state of the module such as:

```
initial_max_stream_data.is_set(fml:s)
initial_max_stream_data.value(fml:s)
initial_max_data.is_set(fml:s)
initial_max_data.value(fml:s)
trans_params(scid) := tps.transport_parameters
                         .value(idx)
                         .set(trans_params(scid));
```

C. Monitor

Before deeping into the specification statements, one important concept in Ivy is the monitor. The QUIC specification is based on that concept. Monitors are useful to separate the specification of an object from the object itself. Monitors are objects in Ivy that contains the specification but that doesn't execute them.
It allows to control the flow of the specifications' executions. Let's see this example :

```
around app_server_open_event {
    require conn_requested(dst,src,dcid);
    require ~connected(dcid) & ~connected(scid);
    ...
    call map_cids(scid,dcid);
    call map_cids(dcid,scid);
    ack_credit(scid) := ack_credit(scid) + 1;
}
```

Let's note that around is a shortcut keyword. The content above is the same as :

```
before app_server_open_event {
    require conn_requested(dst,src,dcid);
    require ~connected(dcid) & ~connected(scid);
}
after app_server_open_event {
    call map_cids(scid,dcid);
    call map_cids(dcid,scid);
    ack_credit(scid) := ack_credit(scid) + 1;
}
```

We can see the use of two important keywords : `before` and `after`. What is inside the before statement will be executed each time there is an event `app_server_open_event`. We will thus in this example require that there is a connection request and that the the client's `cid` for this connection as well as the `cid` selected by the server for this connection are not connected. The after statement is similar : it execute is content each time an `app_server_open_event` is generated.

This is a shortcut of this code that produce the same result :

```
action check_connection {
    require conn_requested(dst,src,dcid);
    require ~connected(dcid) & ~connected(scid);
}
action check_mapping {
    call map_cids(scid,dcid);
    call map_cids(dcid,scid);
    ack_credit(scid) := ack_credit(scid) + 1;
}

before app_server_open_event execute check_connection;
before app_server_open_event execute a2;
```

```
instance pc : pcap(quic_packet,quic_deser)

action show_packet(src:ip.endpoint,
                   dst:ip.endpoint,
                   pkt:quic_packet)
import show_packet

action packet_event(src:ip.endpoint,
                    dst:ip.endpoint,
                    pkt:quic_packet) = {}

action tls_send_event(src:ip.endpoint,
                      dst:ip.endpoint,
                      scid:cid,dcid:cid,
                      data:stream_data) = {}

implement pc.handle(src:ip.endpoint,
                    dst:ip.endpoint,
                    pkt:quic_packet) {
    # print the packet
    call show_packet(src,dst,pkt);
    # infer any TLS events
    call infer_tls_events(src,dst,pkt);
    # monitor the protocol
    call packet_event(src,dst,pkt);
}
```

D. Control and expression

Here we define control flow statement such as sequential execution, calling actions, conditions and loops. Ivy also define expression which are represented by first order logic operator: and (`&`), or (`|`), implies (`->`), iff (`<->`) and equality (`=`) and negation (`~`). Let's see a QUIC example for that:

```
object frame = {
    ...
    object crypto = {
        ...
        action handle(f:frame.crypto,
                      scid:cid,
                      dcid:cid,
                      e:quic_packet_type)

        around handle {
            #Implication
            require num_queued_frames(scid) > 0
                    -> e = queued_level(scid);
            #negation
            require ~conn_closed(scid);
            require ~(e = quic_packet_type.zero_rtt);
            #inequality
            require (f.offset + f.length)
                    <= crypto_data_end(scid,e);
            # /!\ equality /!\
            require f.data = crypto_data(scid,e)
                .segment(f.offset,f.offset+f.length);
            ...
            # Value assigment
            var length := f.offset + f.length;
            # if statement
            if crypto_length(scid,e) < length {
                crypto_length(scid,e) := length
            };
            var idx := f.offset;
            # Loop: should not be used for initialisation
            # (placeholder concept instead) TODO
            while idx < f.offset + f.length {
                crypto_data_present(scid,e,idx) := true;
                idx := idx.next
            };
            call enqueue_frame(scid,f,e,false);
            if e = quic_packet_type.handshake {
                established_1rtt_keys(scid) := true;
            }
        }
    }
}
```

The sequential execution is defined by `;` character, every time we expect another requirement or action after a statement we put this character. We can call an action with the keyword `call` which execute its code.

E. Non-deterministic choice

```
action handle_tls_extensions
    (src:ip.endpoint,
     dst:ip.endpoint,
     scid:cid,
     exts:vector[tls.extension],
     is_client_hello:bool) = {
    # We process the extensions in a message in order.
    var idx := exts.begin;
    while idx < exts.end {
        var ext := exts.value(idx);
        # For every `quic_transport_parameters` extension
        if some (tps:quic_transport_parameters) ext *> tps {
            call handle_client_transport_parameters(src,
                      dst,scid,tps,is_client_hello);
            trans_params_set(scid) := true;
        };
        idx := idx.next
    };
}
```

Non-deterministic choice can be represented with `*` and used for two situations:

1) On the right-hand side of an assignment, which means that the variable can take any value possible of the type.
2) In condition to create non-deterministic branch, in the presented example this represent the fact that the

extension supported by client and server are non-deterministic and said more or less ”if there is at least one transport parameter available in one of TLS extension proposed, then we call the action `handle_client_transport_parameters`” meaning that the proposed extension is non-deterministic in some way since it change from implementation to implementation.

F. `require` and `ensure`

Primitive actions of Ivy, only `require` is really used for QUIC specification. The actions `ensure` and `require` are similar in the sens that if the condition return false, then the action fails and so the execution. The difference is that for `ensure` it is to the action to make the condition true.

We already seen example of `require` which is related to **pre-condition**, so here is one for `ensure` which is used to represent **post-condition**, this example has been taken from the Ivy documentation:

```
action decr(x:t) returns (y:t) = {
    require x > 0;
    y := x - 1;
    ensure y < x;
}
```

IV. QUIC REPRESENTATION

A. QUIC connection protocol

The main file for representing the QUIC connection protocol can be found at `examples/quic/quic_connection.ivy` of the branch `quic18_client` branch of McMillan since this is the last version that work for us. By the way, note that all the comments and variable’s names are not updated.

This file describe and define all states and actions that are used during the QUIC connection and is defined in function of the following sub-protocol part.

1) `examples/quic/quic_types.ivy` :

In this file we find the all the definition of elementary common type that are used in the QUIC specification such as Connection ID type `cid`, the protocol version `version`, the packet number `pkt_num` and many other ones.

2) `examples/quic/quic_frame.ivy` :

Now we define a type object to represent a general QUIC frame. Then there are definitions of all other QUIC variant frames with their updated state and an associated action on how to handle them.

3) `examples/quic/quic_packet.ivy`:

This file is very simple and contain a type object representing a packet, defined by the packet’s field of the QUIC specification.

4) `examples/quic/tls_protocol.ivy` :

Describe the TLS v1.2 handshake protocol and also the transfer of raw data but is not fully general, some features cannot be represented for now such as allowing TLS messages that span multiple TLS records. There are also a serializer (used to convert Ivy object into stream of byte inside the memory with c++) and deserializer (which is the reverse process).

5) `examples/quic/quic_transport_parameters.ivy`:

The different QUIC transport parameters are represented as different modules differentiate with their respective field and value.

B. QUIC packet protection

In `examples/quic/quic_protection.ivy` we can find at first representation of possible packet field that are always readable even if the packet is protected. Then there is the definition of the encryption and decryption processes and all actions needed. It contains also a serializer and deserializer to interact with memory directly.

C. QUIC Ivy serializer/deserializer

Here again, there are two important files that are used to put and get states/object of the QUIC protocol (except for the preceding concept where there is also this) directly into the memory.

D. Others

There is also a representation for the UDP implementation and TLS messages and many other sub-concepts that can be linked to QUIC of course but citing all them would be very long but what we can say is that the general structure of all these specification is quite similar with sometimes one new Ivy concepts used or not depending on the situation.

What we can conclude here is that Ivy is quite interesting in the sens that it is quite logical and incremental to represent any protocol specification. You create building blocks that fit into each other, quite similar in how we could do for OOP programming which abstract the details very well. However there is usually a need for serializer/deserializer which is more procedural since we interact with memory so this counter balance the last point.

There is also the fact that code rely on so many sub-protocol specification that inspecting and knowing from where a possible mistake could come from is really difficult.

V. Decidability algorithm and concept

Since the dawn of time, verifying and proving theorems and logical rules is very complex and time consuming tasks. This is why it is imperative to have efficient algorithm and for that, McMillan and his teams decided to use the Microsoft's Z3 theorem prover. [9]

A. Logic concepts

First of all, we will define some logic concepts that are needed in order to understand how Ivy works.

1) First-order logic:

Or predicate logic, where the main differences with propositional logic are:

1) Propositions are replaced by predicates;
2) Existential and universal quantifiers are introduced
3) It is possible to express relationships between the elements of a set in a concise way, even when the set is of infinite size

With this logic we can have expression such that ”All men are mortal, Socrate is a man so Socrate is mortal”.

2) Weakest preconditions:

A solver must be able to transform the predicate from the first order logic to be able to infer the sentence. There are two main ways to build valid deduction : weakest-precondition and strongest-postconditions. Ivy construct his proofs by using weakest liberal precondition. The most general (i.e., weakest precondition) P that satisfies [12]

$$\{P\}\ S\ \{Q\} \equiv \text{ Hoare Triple}$$

With S a statement, P the precondition and R the postcondition of the statement. To check $\{P\}S\{Q\}$, we need to prove that $P \rightarrow wp(S, Q)$.

Here is a small resume of some of the rules:

$$wp(x := E, Q) = Q[E/x]$$

$$wp(\text{assert } P, Q) = P \wedge Q$$

$$wp(\text{assume } P, Q) = P \rightarrow Q$$

$$wp(s1; s2, Q) = wp(s1, wp(s2, Q))$$

$$wp(s1\ s2, Q) = wp(s1, Q) \wedge wp(s2, Q)$$

3) Weakest liberal preconditions:

Weakest liberal precondition is denoted as wlp. If S is a program statement and Q is a some condition on the program state, $wlp(S, Q)$ is the weakest condition P such that, if P holds before the execution of S and if S terminates, then R holds after the execution of S. [11] [13]

Again, for each triple $\{P\}S\{Q\}$, we have to prove that $P \rightarrow wps(S, Q)$ with the modified rules.

The difference with a weakest precondition wp is that in the liberal one we don't guarantee that S terminates.

4) WLP calculus and rules:

$$wlp(skip, Q) = Q \textbf{ [1]}$$

$$wlp(X := E, Q) = Q[E/X] \textbf{ [2]}$$

($Q[E/X]$ is the substitution of every occurrence of X in Q state by the value E)

Ivy example :

- Let's have a Q state containing $x = 4$ and $y = 8$
- $wlp(x := y, Q) = Q\{x = 8, y = 8\}$

$$wlp(s1; s2, Q) = wlp(s1, wlp(s2, Q)) \textbf{ [3]}$$

This rule explain why we are going backward when we use weakest precondition: the first WLP calculus rule that will be effectively applied will start by the last statement.

Ivy example :

- We want to prove that Y = 8.
- $wlp(x := 3; y = 5; y := x + y, Q) = Q = \{y = 8\}$
- $wlp(x := 3; y = 5, wlp(y := x + y, Q) = Q = \{y = 8\}$
- $wlp(x := 3; wlp(y = 5, wlp(y := x + y, Q)) = Q = \{y = 8\}$
- $wlp(x := 3, wlp(y = 5, Q)) = Q = \{x + y = 8\}$
- $wlp(x := 3; Q3) = Q = \{x + 5 = 8\}$
- $Q = \{8 = 8\}$

It is obvious that 8=8 we therefore proved that $y = 8$.

$$wlp(\text{if } C \text{ then } s1 \text{ else } s2, Q) =$$
$$(C \rightarrow wlp(s1, Q)) \wedge (\neg C \rightarrow wlp(s2, Q)) \textbf{ [4]}$$

$$wlp(\text{assume } S, Q) = (S \rightarrow Q) \textbf{ [5]}$$

$$wlp(\text{assert } S, Q) = (S \wedge Q) \textbf{ [6]}$$

5) Used notation:

B. Quantifier-free linear integer arithmetic

Algorithms that solve the problem to say if one first-order logic formula (without quantifier) are exponential in time for the worst cast since this problem is NP-complete.

These algorithms handle a theory called ”Quantifier-Free Linear Integer Arithmetic” (QFLIA). [14] It considers a set of variable X only containing integers which allows us to form combination of constraints define with operators ”`and`” ($\wedge$), ”`or`” ($\vee$) and ”`not`” ($\neg$).

It uses a kind of proof by contradiction where as reminder, to prove proposition $P \to Q$ then:

- We assume P to be false, i.e $\neg P$
- We have to show that $\neg P \to (Q \land \neg Q)$ which is a contradiction, so P must be true.

The algorithm proceed more or less the same way by searching if the negated proposition/condition has a solution or not meaning in the first case that the proposition is not valid.

C. Decidable fragment and Z3

First let's explain what is a decidable fragment. A fragment in mathematical logic is a subset of this logic theory which is obtain by adding restriction on it. We have a reduction from a problem to another one.

Proceeding like that allow to take advantage from the property that the time complexity of saying whether a fragment is satisfiable/decidable or not cannot be higher than for the original problem.

Z3 SMT solver use this and given enough time and memory, it should always be able to determine if a formula in the fragment is satisfiable.

In practice, Z3 is very predictable, constant for formula inside fragment than formula outside because it can easily have a reliable counter-models/negated proposition. It is also very efficient for small formula as we can expected.

We also know that all quantifier-free formulas are in decidable fragment. This allow us to reason on formula with quantifier, for that let's take a look at a simple but good example of McMillan:

$$\forall X : f(X) > X$$

To prove this, we can instantiate the quantifier for some constant y like this:

$$f(y) > y$$

We know as a consequence of Herbrand's theorem that the method of instantiate quantifier is a complete method, so if we show that the verified condition is not satisfiable using instanciation, then this formula is unsatisfiable in general. But the inverse is not always true except for decidable fragment where it can be shown that there is always a finite set of instances such that is they are satisfiable then the formula is satisfiable.

References

[1] (n.d.). Retrieved from http://microsoft.github.io/ivy/language.html
[2] (n.d.). Retrieved from https://www.eecs.berkeley.edu/ apanda/
[3] (n.d.). Retrieved from https://www.cs.tau.ac.il/ msagiv/
[4] Axel Legay. (n.d.). Retrieved from https://uclouvain.be/fr/repertoires/axel.legay
[5] Pr. Olivier Bonaventure: INL: IP Networking Lab. (n.d.). Retrieved from https://inl.info.ucl.ac.be/obo.html
[6] http://kenmcmil.com/
[7] http://www2.mta.ac.il/ sharon.shoham/
[8] http://people.mpi-inf.mpg.de/ mvoigt/files/Dissertation_with_hyperref.pdf
[9] https://github.com/Z3Prover/z3
[10] "Guarded commands, nondeterminacy and formal derivation of programs"
[11] (n.d.). Retrieved from http://microsoft.github.io/ivy/decidability.html
[12] https://tinyurl.com/y6z2wv4r
[13] https://www.lri.fr/ marche/MPRI-2-36-1/2013/poly1.pdf
[14] https://cs.nyu.edu/media/publications/king_tim.pdf [p105]
[15] https://github.com/microsoft/ivy
[16] https://link.springer.com/chapter/10.1007/978-3-030-53291-8_12 [p190]